

Code-Layout, Readability and Reusability

Date	29 June 2026
Team ID	
Project Name	A comprehensive measure of well being
Team Members	Chikali Sarika, Anusha Gollapalli, Karishma Sharon Kaparapu, Ambika Chitikela, Parneeth Devanaboina
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the HDI Predictor codebase in terms of its structure, readability, and reusability. The project is organized into clearly separated data preprocessing, model training, prediction, and Flask web layers, with consistent naming and modular functions that support future extension and reuse.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP 8 style followed across all Python modules
2	Proper File Structure	Files and folders are logically organized	Yes	Separated into data/, model/, templates/, static/, and app.py
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g., life_expectancy, schooling_years, gni_per_capita, hdi_tier
4	Function / Method Names	Functions are descriptively named	Yes	e.g., preprocess_data(), train_model(), predict_hdi_tier()
5	Code Comments	Inline and block comments explain logic	Partial	Core preprocessing and model training scripts are commented; Flask routes need more inline notes
6	Modular Design	Code is split into reusable functions/modules	Yes	Preprocessing, training, and prediction kept in separate modules
7	No Redundant Code	Duplicate or unused code is removed	Yes	Notebook experimentation cells cleaned up before moving logic into scripts

8	Error Handling	Exceptions and errors are handled gracefully	Partial	Input validation implemented on the form; broader try/except coverage for file I/O still in progress
---	----------------	--	---------	--

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	data_preprocessing.py	Python, Pandas, NumPy	Cleaning and preparing UNDP/Kaggle HDI datasets before training and before each new prediction request	High
2	train_model.py	Python, scikit-learn	Training and re-training the HDI classification model whenever new data is added	High
3	predict.py	Python, scikit-learn, Pickle	Loading the saved model and generating predictions for the Flask app and any future API endpoint	High
4	visualization_utils.py	Python, Matplotlib, Seaborn	Generating indicator distribution and feature-importance charts in both the notebook and the web app	Medium
5	app.py (Flask routes)	Python, Flask	Handling form submissions and rendering prediction results across all three project scenarios	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear separation between data processing, model training, and the Flask web layer
Readability	4	Descriptive naming conventions and consistent formatting across modules
Reusability	4	Preprocessing and prediction functions are reused across the notebook, CLI, and web app
Documentation / Comments	3	Key functions are commented; README and inline docs to be expanded further
Overall Score	4	Solid, maintainable codebase with minor gaps in error handling and comments